
作业报告

课程名称： 算法设计与分析

作业题目： 粉刷栅栏

专业：

班级：

学号：

姓名：

联系方式：

邮箱：

2025 年 10 月 15 日

1.概述思路

题目理解:

我们有一排长度为 n 的栅栏，栅栏的高度依次为 $h[i]$ 。我们需要给这些栅栏上漆。每次可以选择：

1.涂刷一行：即选择一个高度，将所有未上漆且高度等于该高度的位置全部涂刷。

2.涂刷一列：即选择一个位置，将该位置从底部开始涂刷到其最高处。

目标是最少涂刷次数，使得所有栅栏都被上漆，而且不能多刷（即不能在已经上漆的位置再次上漆）。

暴力解法:

最直接的方法是**逐个列涂刷**，也就是对每个栅栏都进行垂直涂刷。这样，总共需要 n 次涂刷。但是这种方法没有利用到横向涂刷的优势。暴力解法的复杂度是 $O(n)$ ，即对每个栅栏进行一次操作。

为了优化，考虑使用分治的思想。

基本思路:

1.找最小高度：在当前区间 $[l, r]$ 中，找到最小高度 $minH$ 。

2.横向涂刷 or 纵向涂刷：将所有栅栏从当前基准高度 $baseH$ 涂刷到 $minH$ ，需要的涂刷次数为 $minH - baseH$ ；同时考虑纵向涂刷所需的操作次数 $r - l + 1$ 比较哪个更少。

3.递归处理剩余部分：对于高度高于 $minH$ 的部分，递归地进行上述步骤。

4.返回结果：对每次递归返回时选择次数少的涂刷方法

具体步骤:

1.函数定义：设函数 $paint_fences(l, r, baseH)$ 表示从位置 l 到 r 的栅栏，当前已经涂刷到高度 $baseH$ ，返回值是所需的最少涂刷次数。

2.递归公式:

$$paint_fences(l, r, baseH) = \min\{paint_column, paint_row + \sum paint_fences(new_l, new_r, minH)\}$$

其中， $minH$ 是 $h[l..r]$ 的最小值， new_l ， new_r 是高度大于 $minH$ 的连续区间的起始和结束位置。

$paint_column$ 是纵向涂刷所需的次数。

3.解释:

(1) 纵向涂刷($paint_column$): 总共需要 $r - l + 1$ 次。

(2) 横向涂刷到最小高度并递归处理 $paint_row + \sum paint_fences(new_l, new_r, minH)$: 先横向涂刷到最小高度，需要 $minH - baseH$ 次，然后对每个高度大于 $minH$ 的子区间递归处理。

复杂度分析:

1.时间复杂度:

(1) **最坏情况**: 对于每个位置，都可能递归到深度为 $h[i]$ （因为每次高度至少减少1），因此复杂度近似为 $O(n \times hmax)$ 。

(2) **最佳情况**: 如果所有栅栏高度相同，则只需要一次横向涂刷，复杂度为 $O(n)$ 。

在本题中， $h[i] \leq 10$ ，因此 $hmax$ 很小，算法的复杂度近似为 $O(n)$ 。

2.空间复杂度: 递归调用的栈空间，最深为 $hmax$ ，因此空间复杂度为 $O(hmax)$ 。

2.代码

```
#include <iostream>
#include <string>
#include <math.h>
#include <numeric>
using namespace std;

int n, h[5010]; // 全局变量 n 表示栅栏的数量, h[] 数组存储每个栅栏的高度 (索引从 1 开始)

// 递归函数, 计算从位置 left 到 right 栅栏, 在已涂刷高度为 baseH 的情况下, 所需的最少涂刷次数
int paint_fences(int left, int right, int baseH) {
    // 基本情况: 如果区间无效, 返回 0, 不需要额外的涂刷
    if (left > right) {
        return 0;
    }

    // 初始化变量
    int minH = 11; // 当前区间的最小高度, 初始化为 11 (题目中栅栏高度最大为 10)
    int pos_min = 0; // 最小高度所在的位置
    int paint_column = 0; // 垂直涂刷次数 (逐个栅栏涂刷)
    int paint_row = 0; // 水平涂刷次数 (从 baseH 涂刷到 minH 所需的次数)

    // 遍历区间, 找到最小高度 minH, 位置 pos_min, 并计算垂直涂刷次数 paint_column
    for (int i = left; i <= right; i++) {
        // 更新最小高度和其位置
        if (minH > h[i]) {
            minH = h[i];
            pos_min = i;
        }
        // 如果当前栅栏高度大于已涂刷高度, 需要涂刷
        if (h[i] > baseH) {
            paint_column++; // 垂直涂刷次数加一
        }
    }

    // 计算水平涂刷到 minH 所需的次数
    paint_row = minH - baseH;

    // 递归处理最小高度位置左侧和右侧的子区间, 新的已涂刷高度为 minH
    int left_min = paint_fences(left, pos_min - 1, minH);
    int right_min = paint_fences(pos_min + 1, right, minH);

    // 计算选择水平涂刷的总涂刷次数
    int total_paint_row = paint_row + left_min + right_min;
```

```
// 返回垂直涂刷和水平涂刷方案中较小的次数
return min(paint_column, total_paint_row);
}

int main() {
    // 输入栅栏的数量
    cin >> n;
    // 输入每个栅栏的高度，索引从 1 开始
    for (int i = 1; i <= n; i++) {
        cin >> h[i];
    }
    // 调用递归函数，初始区间为[1,n]，已涂刷高度为 0
    cout << paint_fences(1, n, 0);
}
```

3.测试结果

样例测试：

输入如下：

```
5
2 2 1 2 1
```

运行结果：

```
5
2 2 1 2 1
3
```

执行过程：

1.初始调用：*paint_fences*(1,5,0)，区间为[1,5]，已涂刷高度为 0。

2.找到最小高度：区间[1,5]的最小高度为 1。

3.计算涂刷次数：

垂直涂刷次数：5（每个栅栏单独涂刷）。水平涂刷次数：1（从高度 0 涂刷到 1）。

4.递归处理子区间：

找到高度大于 1 的子区间：

[1,2]：高度为 2，递归调用*paint_fences*(1,2,1)。

[4,4]：高度为 2，递归调用*paint_fences*(4,4,1)。

5.递归计算：

（1）对于子区间[1, 2]：最小高度为 2，水平涂刷次数为 1，垂直涂刷次数为 2。无需进一步递归，返回最小涂刷次数为 1。

（2）对于子区间[4,4]：最小高度为 2，水平涂刷次数为 1，垂直涂刷次数为 1。无需进一步递归，返回最小涂刷次数为 1。

6.计算总的涂刷次数：

总涂刷次数为 1（水平涂刷到高度 1）+ 1（左子区间）+ 1（右子区间）= 3。

7.选择最小方案：与垂直涂刷次数 5 比较，选择较小的 3。

8.输出结果：最少的涂刷次数为 3。

9.具体如图：



测试用例 1:

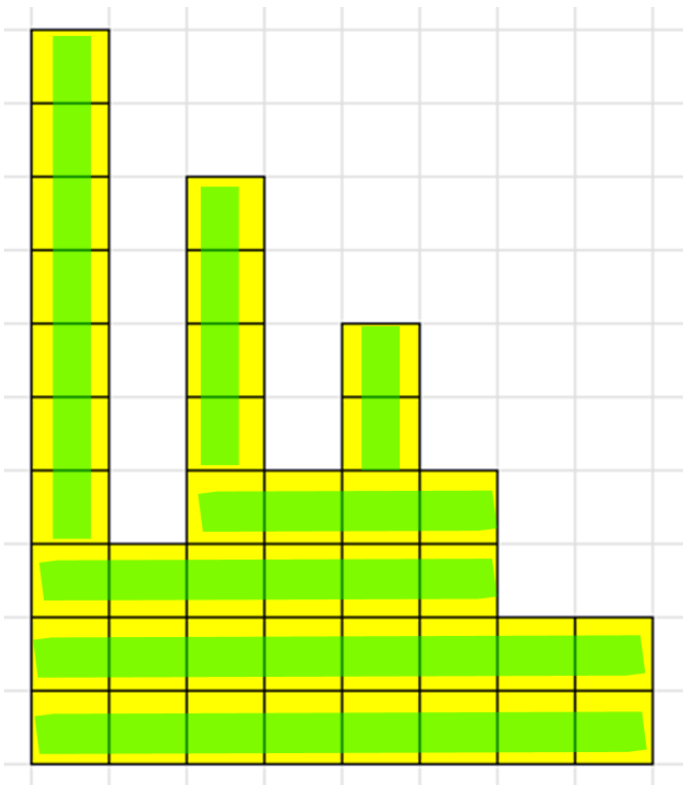
输入如下：

```
8
10 3 8 4 6 4 2 2
```

运行结果：

```
8
10 3 8 4 6 4 2 2
7
```

结果如图：



测试用例 2:

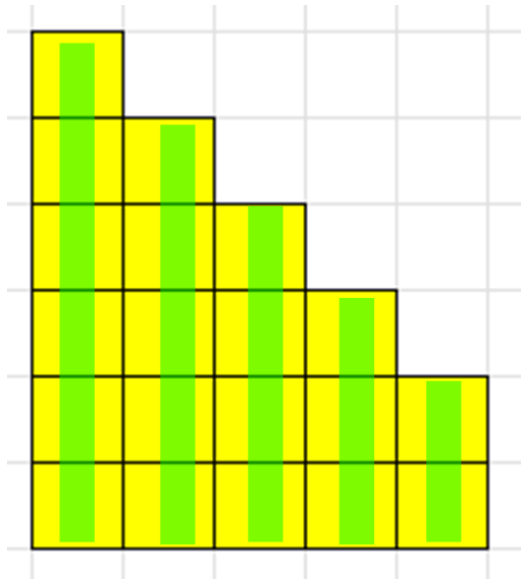
输入如下:

```
5
6 5 4 3 2
```

运行结果:

```
5
6 5 4 3 2
5
```

结果如图:



测试用例 3:

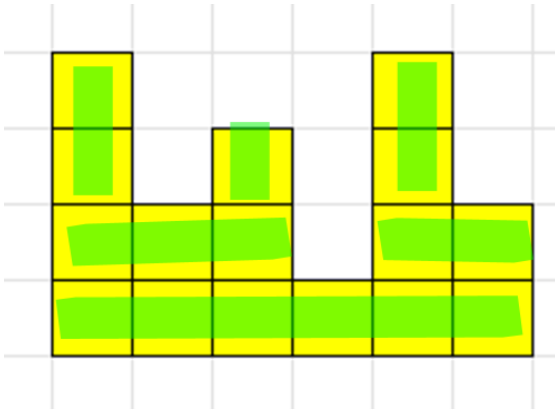
输入如下:

```
6
4 2 3 1 4 2
```

运行结果:

```
6
4 2 3 1 4 2
6
```

结果如图:



测试用例 4:

输入如下:

```
7
10 2 8 2 7 4 6
```

运行结果:

```
7
10 2 8 2 7 4 6
7
```

结果如图:

